Tuning for what the business pays for

Resource-aware program search on real data: the strongest case studies

JopeDime • case-study brief • evidence commit `9db418413a1c` • data snapshot 2026-11-03T16:21:35Z

Status: interim, validation stage. Every result is measured on the cut-0 holdout of each dataset's `train.csv`. The pre-declared single look at the sealed test split has *not* been taken. When it has, `make -C docs/case-study-brief` regenerates this brief, and a claim is final only if that look agrees. Inputs: 480 evidence files, digest `a8f3745e190a0c2f`. Every plotted main run ends exactly at the value its receipt records (107 of 107; Section 4).

Summary

JopeDime's program search can rank programs by a *business objective*: a fitness function that reads what a program costs to train and to serve as well as how accurate it is. A cost can be priced or imposed as a hard limit; a program that breaks a limit is refused, never ranked. We ran 14 named case studies on real corpus datasets against the strongest alternatives we could run on our hosts, all scored by the same product evaluator on the same rows: CatBoost, LightGBM, XGBoost, scikit-learn HistGB, AutoGluon (best quality), the tabular foundation models TabPFN v2 (Prior Labs) and Kumo Tabular (NVIDIA), and Ridge, random forest, extra trees and k-NN.

- **Online fare quoting under a latency budget (case 03).** Evolved programs answer one request in 3.426 µs (p99) at RMSE 0.001 842, level with the best library, XGBoost (0.001 858). Served from Python, every library takes 471–2031 µs per call. The one model that is more accurate, TabPFN v2 (0.001 601), costs 141 ms per row even in batch.
- **Auditable models on 1,142 features (case 09, no leakage flag).** Extended evolution finds programs of 1–2 nodes with RMSE 0.1279: -11.3% against the most accurate alternative, HistGB (0.1442), and better than AutoGluon (0.1524, 50 MB). The holdout is only 47 rows, so the test look matters most here.
- **Hourly refits billed per CPU second (case 02).** Evolution seeded with library-equivalent programs scores 0.2631 on the business objective, -28.6% against the best alternative (XGBoost, 0.3684), at XGBoost's accuracy and a quarter of its training CPU.
- **Retraining before gate closure (case 01), an instructive loss.** Under the 0.49 s retrain limit our best arm (0.009 215) beats XGBoost by -3.7% and CatBoost and AutoGluon cannot retrain in time, but TabPFN v2 is more accurate still (0.008 006): an in-context model has no training to limit, and the objective did not price its 123 ms-per-row prediction.
- **More generations improve the objective.** Chained (extended) evolution lowered the business KPI below the best program of generation 0 in 10 of 11 cases, by a median 14.4% (Section 4).

Where an alternative wins we say so (case 07, an honest near-miss, and the leaderboard in Section 6). Four datasets proved to leak their target; we claim nothing from them (Section 5).

1 What was built

A fitness specification is data (JSON), never code. It can now read *resource variables*:

measured	`fit_seconds`, `fit_cpu_seconds`, `predict_seconds`, `predict_us_per_row`, `predict_rows_per_second`, `row_latency_p50_us`, `row_latency_p99_us`, `fit_peak_heap_bytes`, `predict_peak_heap_bytes`
structural	`model_bytes`, `program_nodes`, `stage_count`, `train_rows`, `hold_rows` (reproducible)

A variable can be a priced term (`{"metric":"resource","name":...}`), a leaf of a custom per-row expression, or a hard limit (`{"metric":"constrained",...,"limits":[...]}`). The program search (`jope pipeline run --fitness`) measures these readings for every candidate and records them in `observations.json`; `verify-receipt --evidence` re-derives the champion from the recorded values. Measured readings depend on the host, so a search that ranks by them evaluates one candidate at a time.

2 How the studies were run

- **Arms** (one budget each: 96 evaluations or 600 s; RNG seeds 7, 11, 13): **A** generation 0 seeded by our deep-learning policy (step-60000), ranked by the business objective; **B** random generation 0, same objective; **C**/**D** as A/B ranked by accuracy only; **E** generation 0 seeded with our grammar's equivalents of the XGBoost, LightGBM and scikit-learn defaults. *Extended* runs chain 3 rounds (5 on the arm64 host) of up to 1000 evaluations or 600 s each; each round restarts from the 11 best feasible programs found so far.

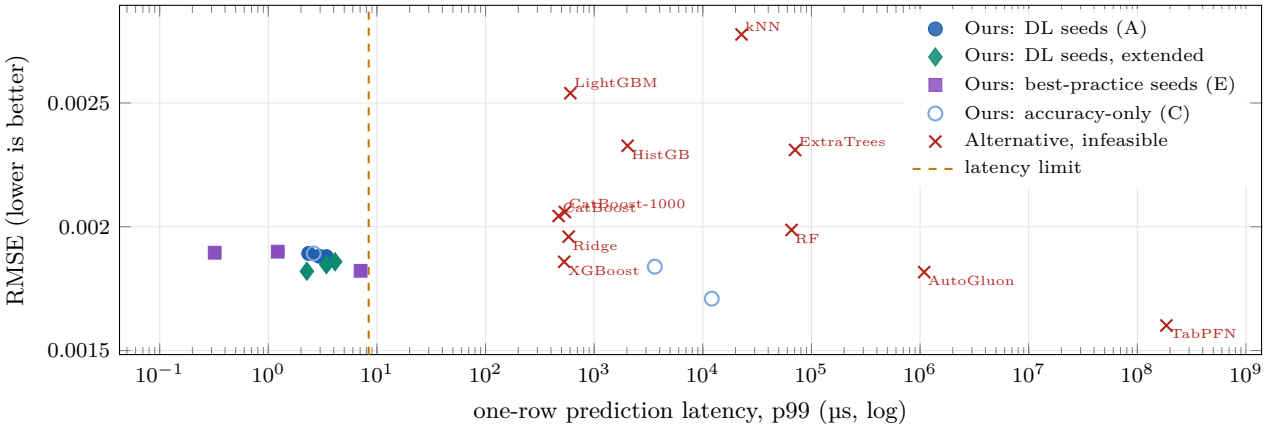
- **Limits and prices** were calibrated before any main run, from a separate calibration run: a limit sits at the median reading, so it binds for about half of the candidates.
- **Fair comparison.** Alternatives fit on exactly the same rows and are scored by the same evaluator (`jopedime fitness evaluate`). A Python-served one-row call is mostly call overhead, so for latency cases every library is also credited with its amortized batch latency (a *native-serving proxy*).
- **Hosts.** xbabe1, xbabe2 (identical Threadripper PRO 3995WX); all arms of a case ran on one host. The arm64 Mac xbabe4 (Linux container) ran only extended chains of a structure-priced case (09), whose objective does not depend on the hardware.

3 The strongest case studies

3.1 Case 03 — online fare quoting under a latency budget

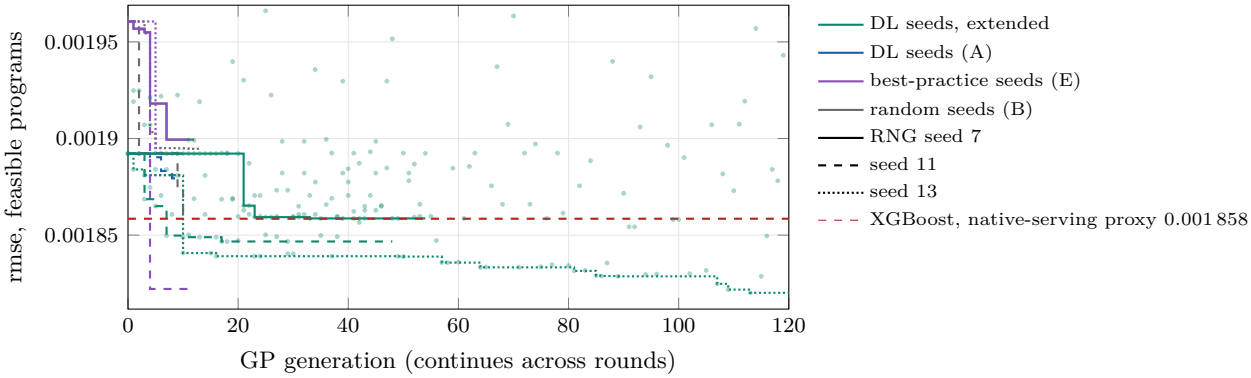
Business question

Which fare model is most accurate while quoting a single trip within a p99 latency budget? Data: NYC Green Taxi Trip Sample with Fare and Distance (10000 rows, 11 features; selection holdout 1630 rows). Objective: RMSE subject to row_latency_p99_us $\leq$ 8.4 µs.



Off the chart (worse than 2× the best): Kumo 0.011 60.

	RMSE	p99 latency (µs)	feasibility
Ours: DL seeds (A, 96 evals)	0.001 884	2.940	3 runs
Ours: DL seeds, extended (3 rounds)	0.001 842	3.430	3 runs
Ours: best-practice seeds (E, 96 evals)	0.001 872	1.220	3 runs
Ours: accuracy-only ranking (C)	0.001 813	3.620×10^{3}	3 runs
TabPFN v2 (Prior Labs, CPU)	0.001 601	1.850×10^{8}	infeasible (0/1)
AutoGluon (best_quality, 300 s)	0.001 817	1.090×10^{6}	infeasible (0/1)
XGBoost (default)	0.001 858	530.0	infeasible (0/3)
Ridge (CV)	0.001 960	582.0	infeasible (0/3)
Random forest (300)	0.001 987	6.560×10^{4}	infeasible (0/3)
CatBoost (default)	0.002 044	471.0	infeasible (0/3)
CatBoost (1000 trees, depth 6)	0.002 060	537.0	infeasible (0/3)
Extra trees (300)	0.002 310	7.070×10^{4}	infeasible (0/3)
scikit-learn HistGB	0.002 327	2.030×10^{3}	infeasible (0/3)
LightGBM (default)	0.002 540	604.0	infeasible (0/3)
k-NN (k=10)	0.002 777	2.270×10^{4}	infeasible (0/3)
Kumo Tabular small (NVIDIA, CPU)	0.011 59	5.370×10^{6}	infeasible (0/1)



Best-so-far (the global best the search holds) of the case's business objective after each generation; faint dots are each generation's best new program in the extended chains. Lines end where a run's budget (96 evaluations or 600 s per round) ran out.

What it shows

- The business objective returns programs at 3.426 µs whose accuracy (0.001 842) is level with XGBoost (0.001 858), the most accurate library, and better than CatBoost, LightGBM and HistGB.
- Ranked by accuracy alone, the search finds programs (0.001 814) that cost 3616 µs per request, about three orders of magnitude over budget. The objective makes that trade explicit.
- The most accurate model measured, TabPFN v2 (0.001 601), needs 141 ms per row even amortized over a batch on CPU: no serving path brings it near 8.4 µs. AutoGluon is 612 MB.

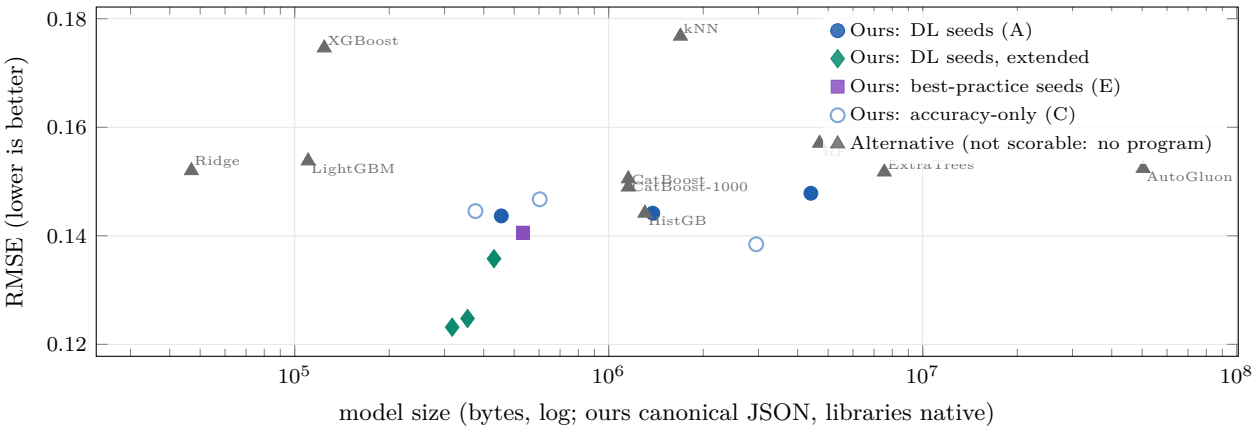
Caveats

With the native-serving proxy the boosted libraries would meet the budget, and the contest becomes accuracy, where our extended champions and XGBoost are at parity. Pickup time is one of 11 features and validation uses random folds (medium leakage risk).

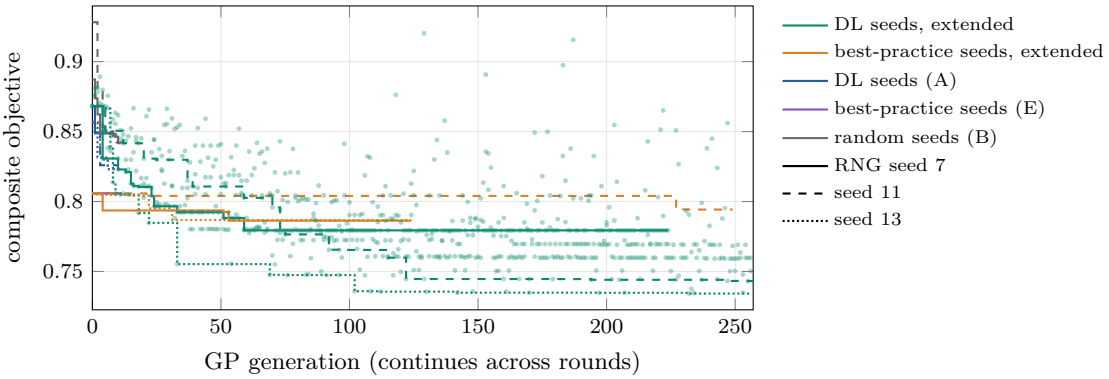
3.2 Case 09 — auditable models on ultra-wide data

Business question

With 1142 features and very few rows, which program balances accuracy against auditability (program size)? Data: OpenML mtp2 (274 rows; cut-0 trains on 181 and selects on 47). Objective: RMSE/0.18 + 0.1 × program nodes/4 (structural, reproducible on any hardware).



	RMSE	model bytes	runs / status
Ours: DL seeds (A, 96 evals)	0.1452	1.380×10^{6}	3 runs
Ours: DL seeds, extended (3 rounds)	0.1279	3.550×10^{5}	3 runs
Ours: best-practice seeds (E, 96 evals)	0.1406	5.330×10^{5}	3 runs
Ours: accuracy-only ranking (C)	0.1432	6.030×10^{5}	3 runs
scikit-learn HistGB	0.1442	1.300×10^{6}	not scorable (no program)
CatBoost (default)	0.1490	1.150×10^{6}	not scorable (no program)
CatBoost (1000 trees, depth 6)	0.1505	1.150×10^{6}	not scorable (no program)
Extra trees (300)	0.1518	7.550×10^{6}	not scorable (no program)
Ridge (CV)	0.1520	4.690×10^{4}	not scorable (no program)
AutoGluon (best_quality, 300 s)	0.1524	5.030×10^{7}	not scorable (no program)
LightGBM (default)	0.1538	1.100×10^{5}	not scorable (no program)
Random forest (300)	0.1571	4.690×10^{6}	not scorable (no program)
XGBoost (default)	0.1746	1.240×10^{5}	not scorable (no program)
k-NN (k=10)	0.1768	1.690×10^{6}	not scorable (no program)



Best-so-far (the global best the search holds) of the case's business objective after each generation; faint dots are each generation's best new program in the extended chains. Lines end where a run's budget (96 evaluations or 600 s per round) ran out. Extended chains here ran 5 rounds on the arm64 host.

What it shows

- Extended evolution reaches RMSE 0.1279 with programs of 1–2 nodes and a median 355 KB: -11.3% against HistGB (0.1442, 1304 KB), and ahead of CatBoost (0.1490) and AutoGluon (0.1524, 50 MB).
- The business KPI kept improving with generations: the extended chains lowered it by a median 14.4% from generation 0.

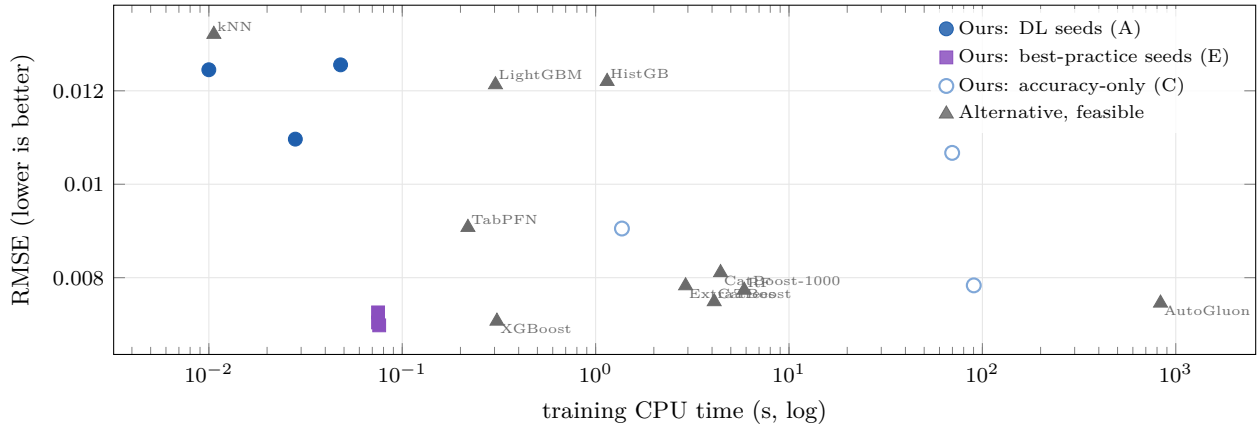
Caveats

The objective prices program structure, which library models do not have, so they are compared on RMSE only. The selection holdout is 47 rows and the extended chains scored thousands of programs on it, so the validation figure is optimistic by construction (selection bias). This case is decided by the test look.

3.3 Case 02 — hourly refits billed per CPU second, and why every cost must be priced

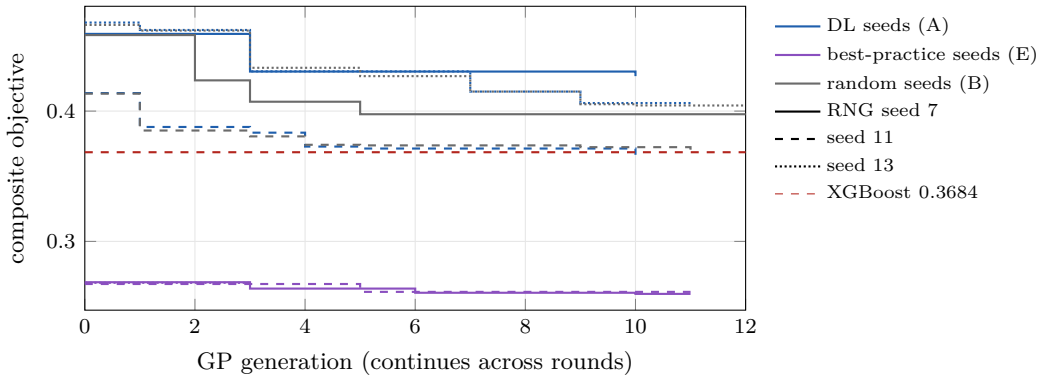
Business question

When every hourly refit is paid in compute, which program minimizes forecast error plus the CPU cost of refitting? Objective: RMSE/σ + 0.1 × fit CPU seconds / median (selection holdout 1039 rows).



Off the chart (worse than 2× the best): Ridge 0.016 40; Kumo 0.024 60.

	RMSE	fit CPU (s)	feasibility
Ours: DL seeds (A, 96 evals)	0.011 99	0.028 00	3 runs
Ours: best-practice seeds (E, 96 evals)	0.007 093	0.075 00	3 runs
Ours: accuracy-only ranking (C)	0.009 186	69.80	3 runs
XGBoost (default)	0.007 070	0.3090	feasible (3/3)
AutoGluon (best_quality, 300 s)	0.007 457	834.0	feasible (1/1)
CatBoost (default)	0.007 487	4.100	feasible (3/3)
Random forest (300)	0.007 733	5.860	feasible (3/3)
Extra trees (300)	0.007 828	2.920	feasible (3/3)
CatBoost (1000 trees, depth 6)	0.008 109	4.430	feasible (3/3)
TabPFN v2 (Prior Labs, CPU)	0.009 076	0.2190	feasible (1/1)
LightGBM (default)	0.012 13	0.3030	feasible (3/3)
scikit-learn HistGB	0.012 20	1.150	feasible (3/3)
k-NN (k=10)	0.013 21	0.010 60	feasible (3/3)
Ridge (CV)	0.016 36	0.013 80	feasible (3/3)
Kumo Tabular small (NVIDIA, CPU)	0.024 57	0.2250	feasible (1/1)



Best-so-far (the global best the search holds) of the case's business objective after each generation; faint dots are each generation's best new program in the extended chains. Lines end where a run's budget (96 evaluations or 600 s per round) ran out.

What it shows

- Seeded with library-equivalent programs, evolution scores 0.2631 on the business objective against XGBoost's 0.3684 (-28.6%): the same accuracy (RMSE 0.007 093 vs 0.007 070) at a median 0.075 s of training CPU against 0.309 s. TabPFN v2 scores 0.3921.
- The lesson: this objective priced *only* training CPU, so one DL-seeded champion is a 40 MB model that predicts slowly. The search was free to trade away a cost nobody priced. An objective is a contract: price every cost the business pays.

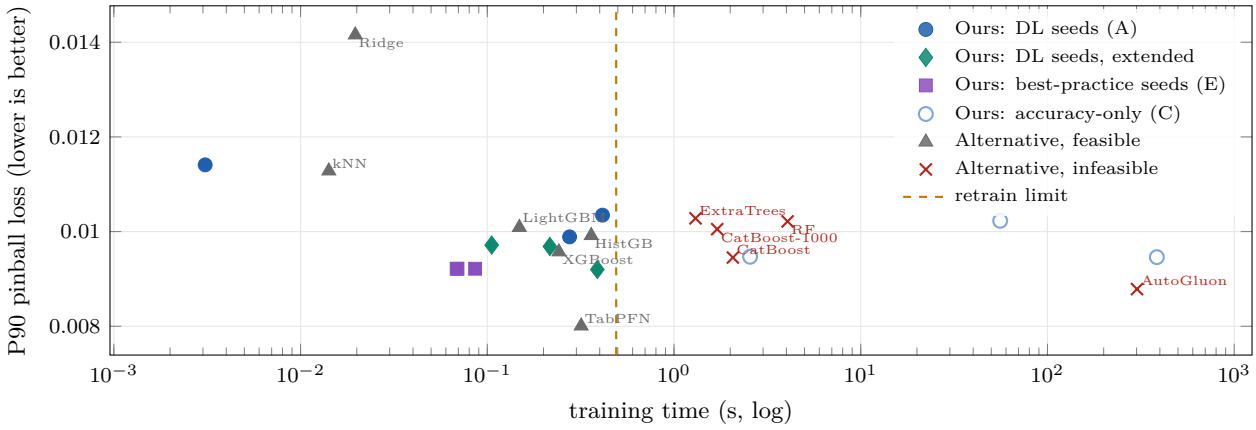
Caveats

Time-ordered data validated on random folds (high leakage risk). The CPU price comes from the calibration rule, not a tariff.

3.4 Case 01 — day-ahead price forecast retrained before gate closure (an instructive loss)

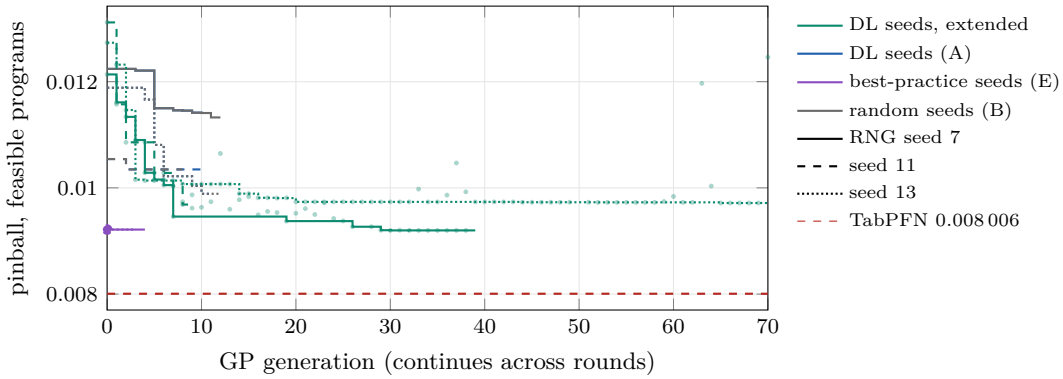
Business question

Which program gives the best P90 day-ahead price forecast that can be retrained inside the window before gate closure? Data: Polish Day-Ahead Electricity Market, Demand, Renewables and Weather (9901 rows; selection holdout 1570). Objective: P90 pinball loss subject to fit_seconds $\leq$ 0.49 s.



Off the chart (worse than 2× the best): Kumo 0.052 50.

	P90 pinball	fit (s)	feasibility
Ours: DL seeds (A, 96 evals)	0.010 55	0.2760	3 runs
Ours: DL seeds, extended (3 rounds)	0.009 533	0.2160	3 runs
Ours: best-practice seeds (E, 96 evals)	0.009 215	0.069 30	3 runs
Ours: accuracy-only ranking (C)	0.009 719	55.80	3 runs
TabPFN v2 (Prior Labs, CPU)	0.008 006	0.3180	feasible (1/1)
AutoGluon (best_quality, 300 s)	0.008 785	302.0	infeasible (0/1)
CatBoost (default)	0.009 452	2.070	infeasible (0/3)
XGBoost (default)	0.009 572	0.2420	feasible (3/3)
scikit-learn HistGB	0.009 917	0.3600	feasible (3/3)
CatBoost (1000 trees, depth 6)	0.010 05	1.700	infeasible (0/3)
LightGBM (default)	0.010 09	0.1480	feasible (3/3)
Random forest (300)	0.010 21	4.060	infeasible (0/3)
Extra trees (300)	0.010 28	1.300	infeasible (0/3)
k-NN (k=10)	0.011 28	0.014 20	feasible (3/3)
Ridge (CV)	0.014 16	0.019 60	feasible (3/3)
Kumo Tabular small (NVIDIA, CPU)	0.052 47	0.3690	feasible (1/1)



Best-so-far (the global best the search holds) of the case's business objective after each generation; faint dots are each generation's best new program in the extended chains. Lines end where a run's budget (96 evaluations or 600 s per round) ran out.

What it shows

- Among tree models, evolution from library-equivalent seeds wins under the limit: 0.009 215 against XGBoost's 0.009 573 (-3.7%). CatBoost (2.1 s) and AutoGluon (302 s, 0.008 785) cannot retrain in time.
- TabPFN v2 is the best feasible model (0.008 006): its "training" only stores the context (0.32 s), and its cost moves to prediction, 123 ms per row on CPU. The objective limited training and priced nothing else, so the result is correct as declared. If the business also bounds forecast time, the objective must say so: the case-02 lesson again.
- Ranked by accuracy alone, our search drifts to programs that need 55.82 s to train (median).

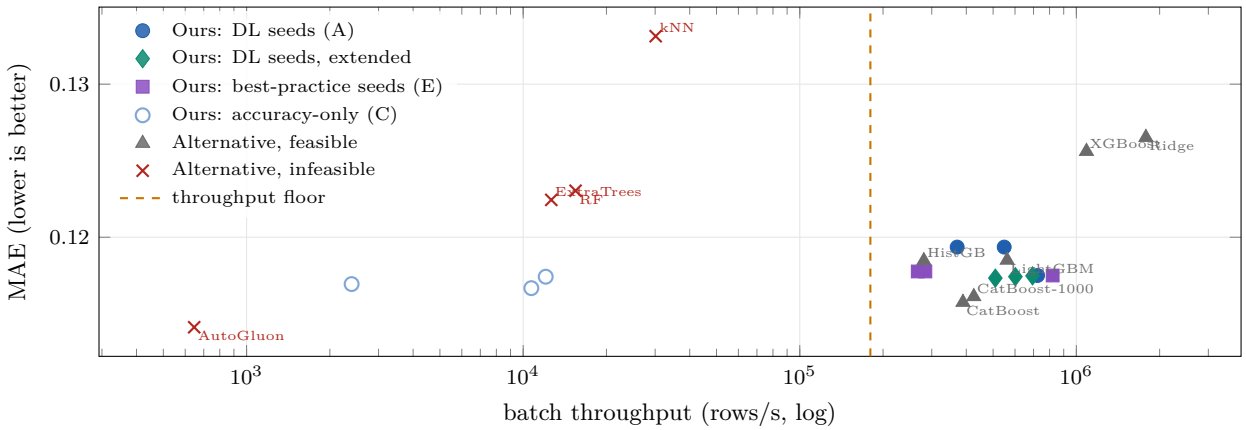
Caveats

Time-ordered market data validated on random folds: this measures in-distribution fit, not forecasting under regime change.

3.5 Case 07 — nightly repricing at a throughput floor (an honest near-miss)

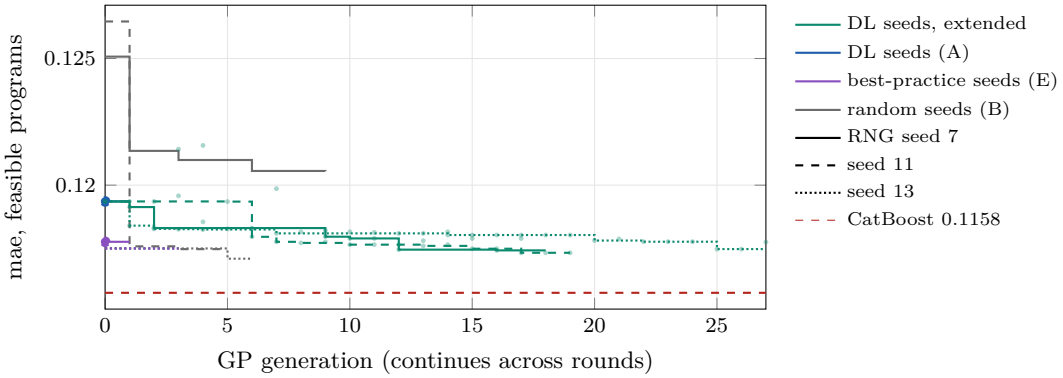
Business question

Which interest-rate model is most accurate (MAE) while repricing the book at no less than the throughput floor? Data: OpenIntro loans_full_schema (Lending Club), interest_rate (10000 rows, 41 features). Objective: MAE subject to predict_rows_per_second ≥ 180000 rows/s.



Off the chart (worse than 2× the best): Kumo 0.2650.

	MAE	rows/s	feasibility
Ours: DL seeds (A, 96 evals)	0.1187	5.480×10^5	3 runs
Ours: DL seeds, extended (3 rounds)	0.1174	6.020×10^5	3 runs
Ours: best-practice seeds (E, 96 evals)	0.1177	2.840×10^5	3 runs
Ours: accuracy-only ranking (C)	0.1170	1.070×10^4	3 runs
AutoGluon (best_quality, 300 s)	0.1141	647.0	infeasible (0/1)
CatBoost (default)	0.1157	3.890×10^5	feasible (3/3)
CatBoost (1000 trees, depth 6)	0.1161	4.260×10^5	feasible (3/3)
scikit-learn HistGB	0.1185	2.810×10^5	feasible (3/3)
LightGBM (default)	0.1185	5.620×10^5	feasible (3/3)
Extra trees (300)	0.1224	1.260×10^4	infeasible (0/3)
Random forest (300)	0.1230	1.550×10^4	infeasible (0/3)
XGBoost (default)	0.1256	1.090×10^6	feasible (3/3)
Ridge (CV)	0.1265	1.780×10^6	feasible (3/3)
k-NN (k=10)	0.1331	3.010×10^4	infeasible (0/3)
Kumo Tabular small (NVIDIA, CPU)	0.2652	170.0	infeasible (0/1)



Best-so-far (the global best the search holds) of the case's business objective after each generation; faint dots are each generation's best new program in the extended chains. Lines end where a run's budget (96 evaluations or 600 s per round) ran out.

What it shows

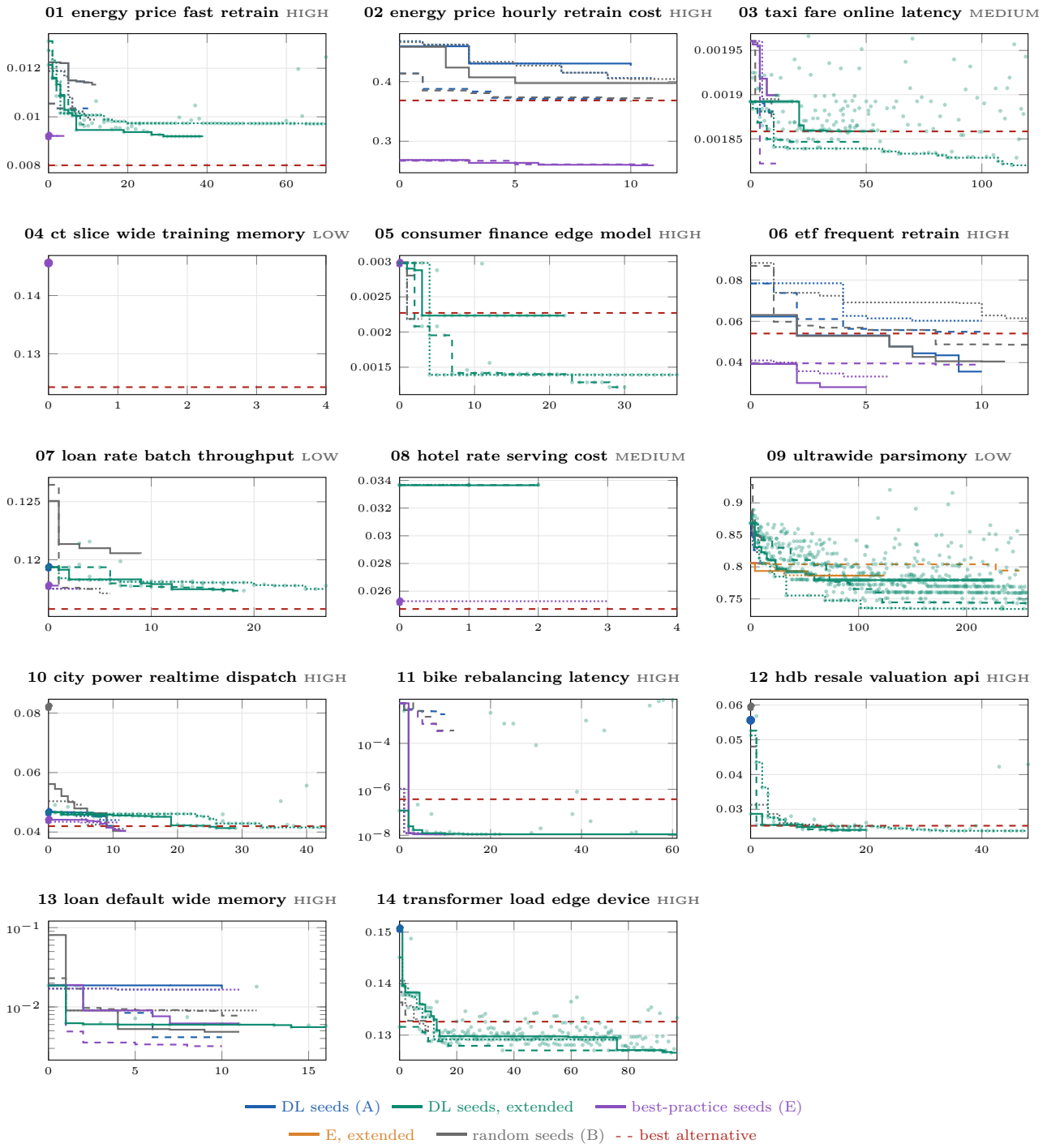
- Ranked by accuracy alone, our search finds a program at 1.070×10^4 rows/s, far below the floor; the business objective returns programs at 5.484×10^5 rows/s.
- CatBoost (0.1158 MAE at 389 100 rows/s) is more accurate and also meets the floor: our best arm is +1.7% (E) to +2.6% (A) behind. On this clean dataset the best library wins, and we report it as such.

4 Evolution on every case

Each panel plots the best-so-far business KPI per generation for every run of the case: colour is the arm, dash the RNG seed (solid 7, dashed 11, dotted 13); the red dashed line is the best alternative under the same objective (native-serving proxy where marked in the leaderboard). The series are recomputed from `observations.json`: for 107 of 107 main runs the final best-so-far equals `receipt.json .search.best.validation_objective`, and for 30 of 32 extended chains it equals the final round's; in the others the global best came from an earlier round and the final round's re-measured champion scored slightly worse (`data/evolution-checks.csv`). The table gives the median gain over seeds from generation 0.

	case	risk	extended (DL seeds)	A (DL seeds)	E (best practice)	B (random)	best alt.
01	energy price fast retrain	HIGH	24.2% (39 gen, 0.009 686)	6.8% (10 gen, 0.010 35)	0.0% (3 gen, 0.009 215)	7.5% (12 gen, 0.010 35)	0.008 006
02	energy price hourly retrain cost	HIGH	–	11.4% (10 gen, 0.4062)	2.3% (11 gen, 0.2613)	13.3% (12 gen, 0.3976)	0.3684
03	taxi fare online latency	MED	2.4% (54 gen, 0.001 847)	0.6% (10 gen, 0.001 881)	3.3% (11 gen, 0.001 895)	3.4% (12 gen, 0.001 895)	0.001 858
04	ct slice wide training memory	LOW	–	–	0.0% (0 gen, 0.1456)	–	0.1243
05	consumer finance edge model	HIGH	53.3% (30 gen, 0.001 391)	0.0% (2 gen, 0.002 982)	0.0% (1 gen, 0.002 982)	26.7% (2 gen, 0.002 186)	0.002 271
06	etf frequent retrain	HIGH	–	29.9% (10 gen, 0.054 93)	19.0% (6 gen, 0.033 24)	35.7% (12 gen, 0.048 65)	0.054 10
07	loan rate batch throughput	LOW	1.6% (19 gen, 0.1174)	0.0% (1 gen, 0.1194)	0.0% (1 gen, 0.1178)	3.6% (6 gen, 0.1175)	0.1158
08	hotel rate serving cost	MED	0.0% (2 gen, 0.033 67)	–	0.0% (1 gen, 0.025 26)	–	0.024 71
09	ultrawide parsimony	LOW	14.4% (256 gen, 0.7431)	3.9% (10 gen, 0.8260)	0.0% (11 gen, 0.8058)	5.1% (11 gen, 0.8417)	–
10	city power realtime dispatch	HIGH	11.0% (29 gen, 0.041 55)	5.7% (10 gen, 0.044 03)	2.0% (11 gen, 0.043 23)	18.3% (12 gen, 0.041 13)	0.041 97
11	bike rebalancing latency	HIGH	90.6% (61 gen, 1.107×10^{-8})	98.9% (10 gen, 1.107×10^{-8})	98.9% (11 gen, 1.107×10^{-8})	98.9% (12 gen, 1.107×10^{-8})	3.692×10^{-7}
12	hdb resale valuation api	HIGH	53.6% (20 gen, 0.023 89)	0.0% (0 gen, 0.055 63)	–	0.0% (1 gen, 0.048 05)	0.025 28
13	loan default wide memory	HIGH	70.2% (16 gen, 0.005 560)	9.6% (10 gen, 0.016 52)	67.3% (11 gen, 0.006 184)	66.3% (11 gen, 0.007 768)	–
14	transformer load edge device	HIGH	11.0% (93 gen, 0.1270)	0.0% (1 gen, 0.1406)	–	4.3% (11 gen, 0.1316)	0.1326

Gain = (best of generation 0 − final global best) / best of generation 0, with the median number of generations and the median final value. For arm E, generation 0 already holds the library-equivalent programs, so its gain is what evolution adds on top of best practice. Runs with 0–2 generations were stopped by the 600 s budget while fitting slow candidates (fits are not yet pre-empted at a limit).



5 What we do not claim

A leakage audit (Ridge R^2, an ordinary least-squares refit and each dataset's column list) demoted four datasets; results on them are not evidence:

- **05 (Survey of Consumer Finances).** The target is INCOME and its components (wages, business, interest, capital gains, retirement, transfers) are features; Ridge reaches $R^2 = 0.993$.
- **11 (bike sharing).** The target is `casual` + `registered`, and both are features.
- **13 (Loan Default Prediction).** LightGBM reaches $R^2 = 0.9994$ and `id` is a feature, consistent with that competition's known leak.
- **06 (ETF prices).** Random folds reward interpolation in time; on a chronological split every model's error rises about tenfold.

Other limits:

- **Selection bias.** Extended chains score thousands of programs on one holdout, so their validation figures are optimistic; only the test look settles them.
- **Instruments.** Our memory reading is heap growth (counting allocator), the libraries' is process RSS growth;

our model size is canonical JSON, theirs native.
- **Foundation models** ran on CPU only. Kumo Tabular (size *small*) has results on cases 01, 02, 03, 06, 07, 08, 10, 11, 12, 14; it was out of memory or not applicable on the others. TabPFN v2 has results on cases 01, 02, 03, 06 so far; the rest are still running.

6 Leaderboard (all 14 cases)

Ranked by the worse of temporal and target-leakage risk, then by our margin over the best alternative under the same business objective (negative = ours better). Ours is the best validation mean among arms A, extended A and E (arm named).

#	case	risk	ours (arm)	best alternative	margin	alt. feasible	accuracy only
1	09 ultrawide parsimony	LOW	0.1279 (extended A, accuracy)	HistGB 0.1442	-11.3%[a]	n/a	-0.6%
2	07 loan rate batch throughput	LOW	0.1174 (extended A)	CatBoost 0.1158	+1.4%	6/11	+2.5%
3	04 ct slice wide training memory	LOW	0.1456 (E)	kNN 0.1243	+17.2%	10/12	–
4	03 taxi fare online latency	MED	0.001 842 (extended A)	XGBoost 0.001 858	-0.9%	6/12[n]	+13.3%
5	08 hotel rate serving cost	MED	0.025 26 (E)	XGBoost 0.024 71	+2.2%	11/11	–
6	13 loan default wide memory	HIGH	0.005 560 (extended A)	none	no feasible alternative	0/12	+15.2%
7	11 bike rebalancing latency	HIGH	1.107×10^{-8} (extended A)	Ridge 3.692×10^{-7}	-97.0%	2/11[n]	-66.1%
8	06 etf frequent retrain	HIGH	0.033 44 (E)	kNN 0.0541	-38.2%	12/12	+135.6%
9	05 consumer finance edge model	HIGH	0.001 614 (extended A)	Ridge 0.002 271	-28.9%	2/11	+19.0%
10	02 energy price hourly retrain cost	HIGH	0.2631 (E)	XGBoost 0.3684	-28.6%	12/12	+29.9%
11	12 hdb resale valuation api	HIGH	0.023 93 (extended A)	XGBoost 0.025 28	-5.4%	4/11[n]	+13.6%
12	14 transformer load edge device	HIGH	0.1276 (extended A)	HistGB 0.1326	-3.8%	3/11[n]	+3.3%
13	10 city power realtime dispatch	HIGH	0.042 59 (E)	LightGBM 0.041 97	+1.5%	2/11[n]	+12.0%
14	01 energy price fast retrain	HIGH	0.009 215 (E)	TabPFN 0.008 006	+15.1%	7/12	+21.4%

[a] Structure-priced objective: compared on accuracy only. [n] Latency case scored with the native-serving proxy. "Accuracy only" compares our accuracy-ranked arm C with the most accurate alternative, all limits ignored (positive = the alternative is more accurate).

Reproducing this brief

- **Cases** in `case_study/`: runners `bin/run_case.sh` and `bin/run_all.sh` (calibrate, main, extended); tables `bin/compare.sh` and `bin/leaderboard.sh`.
- **Evidence** (outside the repository):
  - `~/jope-data/case_study`: receipts, observations and extended rounds;
  - `~/jope-data/resfit-2071/baselines`: the alternatives, scored by `jopedime fitness evaluate`.
- **Regenerate:** `make -C docs/case-study-brief` (runs `build-data.sh`, then `latexmk`).
- **Provenance** of every run the brief reads (pinned `jope` binaries, sha256 prefixes `5e99de09fdfa, ef02d846c14b`: x86-64 and the arm64 build of the same commit):

case	arm	stamp	host	source commit	jope
03	A	20261103T131316Z	xbabe2	9db418413a1c	5e99de09fdfa
03	C	20261103T131316Z	xbabe2	9db418413a1c	5e99de09fdfa
03	E	20261103T152024Z	xbabe1	9db418413a1c + bundle de8542f5c94d	5e99de09fdfa
03	A-extended	extended-20261103T135742Z/s11	xbabe2	9db418413a1c	5e99de09fdfa
03	A-extended	extended-20261103T135742Z/s13	xbabe2	9db418413a1c	5e99de09fdfa
03	A-extended	extended-20261103T135742Z/s7	xbabe2	9db418413a1c	5e99de09fdfa
09	A	20261103T131316Z	xbabe2	9db418413a1c	5e99de09fdfa
09	C	20261103T131316Z	xbabe2	9db418413a1c	5e99de09fdfa
09	E	20261103T152024Z	xbabe1	9db418413a1c + bundle de8542f5c94d	5e99de09fdfa
09	A-extended	extended-20261103T152454Z/s11	xbabe4 (arm64 container)	9db418413a1c + bundle de8542f5c94d+arm64:ef02d846c14b	ef02d846c14b
09	A-extended	extended-20261103T152454Z/s13	xbabe4 (arm64 container)	9db418413a1c + bundle de8542f5c94d+arm64:ef02d846c14b	ef02d846c14b
09	A-extended	extended-20261103T152454Z/s7	xbabe4 (arm64 container)	9db418413a1c + bundle de8542f5c94d+arm64:ef02d846c14b	ef02d846c14b
02	A	20261103T131316Z	xbabe2	9db418413a1c	5e99de09fdfa
02	C	20261103T131316Z	xbabe2	9db418413a1c	5e99de09fdfa
02	E	20261103T152024Z	xbabe1	9db418413a1c + bundle de8542f5c94d	5e99de09fdfa
01	A	20261103T131316Z	xbabe2	9db418413a1c	5e99de09fdfa
01	C	20261103T131316Z	xbabe2	9db418413a1c	5e99de09fdfa
01	E	20261103T152024Z	xbabe1	9db418413a1c + bundle de8542f5c94d	5e99de09fdfa
01	A-extended	extended-20261103T135742Z/s11	xbabe2	9db418413a1c	5e99de09fdfa

case	arm	stamp	host	source commit	jope
01	A-extended	extended-20261103T135742Z/s13	xbabe2	9db418413a1c	5e99de09fdfa
01	A-extended	extended-20261103T135742Z/s7	xbabe2	9db418413a1c	5e99de09fdfa
07	A	20261103T131316Z	xbabe2	9db418413a1c	5e99de09fdfa
07	C	20261103T131316Z	xbabe2	9db418413a1c	5e99de09fdfa
07	E	20261103T152024Z	xbabe1	9db418413a1c + bundle de8542f5c94d	5e99de09fdfa
07	A-extended	extended-20261103T151604Z/s11	xbabe1	9db418413a1c + bundle 0d4599b4e8e6	5e99de09fdfa
07	A-extended	extended-20261103T151604Z/s13	xbabe1	9db418413a1c + bundle 0d4599b4e8e6	5e99de09fdfa
07	A-extended	extended-20261103T151604Z/s7	xbabe1	9db418413a1c + bundle 0d4599b4e8e6	5e99de09fdfa